

Best Coding Practices

Lecture Flow

- Why Best Practices?
- Meaningful Naming
- Writing Modular Code
- Consistent Indentation
- Essential Comments
- The Good the Bad and the Ugly
- Recommendations

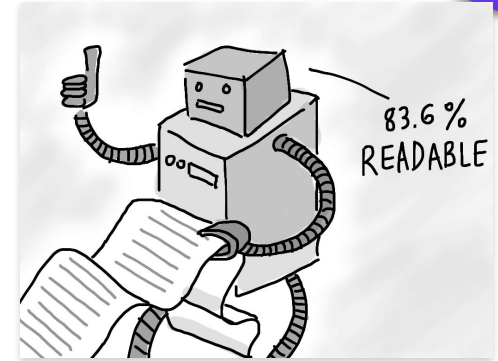


Why Best Practices?

Why Best Practices?

- Do things faster
- Reduce bugs
- Concise code
- Readability

```
while(alive) {  
    eat();  
    code();  
    sleep();  
    repeat();  
}
```



Meaningful Naming

Meaningful Naming

Bad Practice

```
1 class Solution:
2     def isPal(self, s: str) -> bool:
3         l = 0
4         r = len(s)-1
5
6         while l < r :
7             while l < r and not s[l].isalnum():
8                 l += 1
9             while l < r and not s[r].isalnum():
10                 r -= 1
11             if s[l].lower() != s[r].lower():
12                 return False
13             l += 1
14             r -= 1
15
16         return True
17
```

Good Practice

```
class Solution:
    def isPalindrome(self, s: str) -> bool:

        left, right = 0, len(s)-1
        while left < right:
            while left < right and not s[left].isalnum():
                left += 1
            while left < right and not s[right].isalnum():
                right -= 1
            if s[left].lower() != s[right].lower():
                print(s[left], s[right])
                return False
            left += 1
            right -= 1
        return True
```

Meaningful Naming

1. Interviewers seriously care
2. Reduces ambiguity and bugs
3. Helps debugging and readability



Writing Modular Code

Writing Modular Code

Bad

```
class Solution:
    def maxSum(self, grid: List[List[int]]) -> int:
        def calculateSum(top, right):
            curSum = sum(grid[top][right - 2:right + 1]) + grid[top+1][right -1] + sum(grid[top + 2 ][right - 2:right + 1])
            return curSum
        top, right = 0, 2
        maxSum = 0
        while top + 2 < len(grid):
            curSum = 0
            right = 2
            while right < len(grid[0]):
                curSum += calculateSum(top, right)
                right += 1
                maxSum = max(curSum, maxSum)
                curSum = 0
            top += 1
        return maxSum
```

Good

```
class Solution:
    def calculateSum(self, top, right):
        top_sum = sum(grid[top][right - 2:right + 1])
        bottom_sum = sum(grid[top + 2 ][right - 2:right + 1])
        curSum = top_sum + grid[top+1][right -1] + bottom_sum
        return curSum

    def maxSum(self, grid: List[List[int]]) -> int:
        top = 0
        right = 2
        maxSum = 0
        while top + 2 < len(grid):
            curSum = 0
            right = 2
            while right < len(grid[0]):
                curSum += self.calculateSum(top, right)
                right += 1
                maxSum = max(curSum, maxSum)
                curSum = 0
            top += 1
        return maxSum
```

Writing Modular Code

- Helps transiting from solution to code
- Helps seeing the commonalities between similar problems
- Interviewers seriously care
- Reduces bugs
- Helps debugging
- Reusable

Consistent Indentation

Consistent Indentation

Bad Practice

```
class Solution:
    def leastInterval(self, tasks: List[str], items_count: int) -> int:
        counts = [0] * 26
        for i in tasks: counts[ord(i) - ord('A')] += 1
        max_, max_count = max(counts), 0
        for count in counts: max_count += (count == max_)
        return max((max_ - 1) * (items_count+1) + max_count, len(tasks))
```

Good Practice

```
class Solution:
    def leastInterval(self, tasks: List[str], items_count: int) -> int:
        counts = [0] * 26
        for i in tasks:
            counts[ord(i) - ord('A')] += 1
        max_, max_count = max(counts), 0
        for count in counts: max_count += (count == max_)
        return max((max_ - 1) * (items_count+1) + max_count, len(tasks))
```

Consistent Indentation

Bad Practice

```
class Solution:
    def leastInterval(self, tasks: List[str], items_count: int) -> int:
        counts = [0] * 26
        for i in tasks:
            counts[ord(i) - ord('A')] += 1
        max_, max_count = max(counts), 0
        for count in counts: max_count += (count == max_)
        return max((max_ - 1) * (items_count+1) + max_count, len(tasks))
```

Good Practice

```
class Solution:
    def leastInterval(self, tasks: List[str], items_count: int) -> int:
        counts = [0] * 26
        for i in tasks:
            counts[ord(i) - ord('A')] += 1
        max_, max_count = max(counts), 0
        for count in counts:
            if count == max_:
                max_count += 1
        return max((max_ - 1) * (items_count+1) + max_count, len(tasks))
```

Consistent Indentation

Bad Practice

```
class Solution:
    def leastInterval(self, tasks: List[str], items_count: int) -> int:
        counts = [0] * 26
        for i in tasks:
            counts[ord(i) - ord('A')] += 1
        max_, max_count = max(counts), 0
        for count in counts:
            if count == max_:
                max_count += 1
        return max((max_ - 1) * (items_count+1) + max_count, len(tasks))
```

Good Practice

```
class Solution:
    def leastInterval(self, tasks: List[str], items_count: int) -> int:
        counts = [0] * 26
        for i in tasks:
            counts[ord(i) - ord('A')] += 1

        max_, max_count = max(counts), 0
        for count in counts:
            if count == max_:
                max_count += 1

        return max((max_ - 1) * (items_count+1) + max_count, len(tasks))
```

Consistent Indentation

Bad Practice

```
def isValid(row, col, grid):  
    return 0 <= row < len(grid) and 0 <= col < len(grid[0]) and grid[row][col] == "."
```

Good Practice

```
def isValid(row, col, grid):  
    if not (0 <= row < len(grid) and 0 <= col < len(grid[0])):  
        return False  
  
    if grid[row][col] != ".":  
        return False  
  
    return True
```

Consistent Indentation

Bad Practice

```
class Solution:
    def maxSum(self, grid: List[List[int]]) -> int:
        row,col=len(grid),len(grid[0])

        for i in range(row):
            tot=0
            for j in range(col):
                grid[i][j]=tot+grid[i][j]
                tot=grid[i][j]

        ans=0
        for i in range(row-2):
            for j in range(col-2):
                if j>=1:ans=max(ans,grid[i][j+2]-grid[i][j-1]+grid[i+2][j+2]-grid[i+2][j-1]+grid[i+1][j+1]-grid[i+1][j])
                else:ans=max(ans,grid[i][j+2]+grid[i+2][j+2]+grid[i+1][j+1]-grid[i+1][j])

        return ans
```

Consistent Indentation

Good Practice

```
class Solution:
    def maxSum(self, grid: List[List[int]]) -> int:
        row = len(grid)
        col = len(grid[0])

        for i in range(row):
            total = 0
            for j in range(col):
                grid[i][j] = total + grid[i][j]
                total = grid[i][j]

        ans = 0
        for i in range(row-2):
            for j in range(col-2):
                if j>=1:
                    ans= max(ans,
                        grid[i][j+2]-grid[i][j-1]+
                        grid[i+2][j+2]-grid[i+2][j-1]+
                        grid[i+1][j+1]-grid[i+1][j])

                else:
                    ans= max(ans,
                        grid[i][j+2]+grid[i+2][j+2]+
                        grid[i+1][j+1]-grid[i+1][j])

        return ans
```

Consistent Indentation

- Increases code quality and readability
- Interviewers **seriously** care
- Reduces bugs
- Helps debugging

Essential Comments

Essential Comments

```
1 class Solution:
2     def loudAndRich(self, richer: List[List[int]], quiet: List[int]) -> List[int]:
3         people_size = len(quiet)
4         graph = [[] for _ in range(people_size)]
5         indegree = [0 for _ in range(people_size)]
6
7         quieter_person = [person for person in range(people_size)]
8         queue = deque()
9
10        for rich, poor in richer:
11            graph[rich].append(poor)
12            indegree[poor] += 1
13
14        #push nodes with 0 degrees into queue
15        for person in range(people_size):
16            if indegree[person] == 0:
17                queue.append(person)
18
19        while(queue):
20            current_person = queue.popleft()
21
22            for neighbour in graph[current_person]:
23
24                #if parent node having more money is quieter , update
25                if quiet[quieter_person[current_person]] <= quiet[quieter_person[neighbour]]:
26                    quieter_person[neighbour] = quieter_person[current_person]
27
28                indegree[neighbour] -= 1
29
30                if indegree[neighbour] == 0:
31                    queue.append(neighbour)
32
33        return quieter_person
```

Essential Comments

- Helps understanding
- Shows your care to code quality
- Impresses the interviewer

The Good the Bad and the Ugly

```

1 # Check if i, j coordinate is in boundaries of the matrix
2 def isInside(i, j, n, m):
3     if i < 0 or i >= n or j < 0 or j >= m:
4         return False
5     return True
6
7 # Runs dfs and returns true if there is a rectangle
8 def dfs(i, j, n, m, grid, start_i, start_j, start_letter, nodes_count, directions):
9
10     # Mark current cell as visited
11     prev_letter = grid[i][j]
12     grid[i][j] = '*'
13
14     answer = False
15
16     for direction in directions:
17         ni = i + direction[0]
18         nj = j + direction[1]
19
20         # If we hit to the start point, return true
21         if ni == start_i and nj == start_j and nodes_count + 1 >= 4:
22             return True
23
24         # If new explored cell is inside and satisfies condition, go to that cell
25         if isInside(ni, nj, n, m) and start_letter == grid[ni][nj]:
26             answer = dfs(ni, nj, n, m, grid, start_i, start_j, start_letter, nodes_count + 1, directions)
27             if answer:
28                 break
29
30     # Revert back the letter, backtracking
31     grid[i][j] = prev_letter
32
33     return answer
34
35 def main():
36     n, m = list(map(int, input().split()))
37
38     grid = []
39     for _ in range(n):
40         row = [ *input().strip() ]
41         grid.append( row )
42
43     directions = [[0, 1], [1, 0], [-1, 0], [0, -1]]
44
45     for i in range(n):
46         for j in range(m):
47             is_possible = dfs(i, j, n, m, grid, i, j, grid[i][j], 0, directions)
48             if is_possible:
49                 print("Yes")
50                 sys.exit(0)
51
52     print("No")
53
54 if __name__ == "__main__":
55     main()

```

```

1 def dfs(i, j, n, m, g, s_i, s_j, s_l, cnt, dirs):
2     p_l, g[i][j] = g[i][j], '*'
3     ans = False
4     for d in range(1, 5):
5         ni, nj = i + dirs[d], j + dirs[d - 1]
6         if ni == s_i and nj == s_j and cnt + 1 >= 4:
7             return True
8         if (ni >= 0 and ni < n and nj >= 0 and nj < m) and s_l == g[ni][nj]:
9             ans = dfs(ni, nj, n, m, g, s_i, s_j, s_l, cnt + 1, dirs)
10            if ans:
11                break
12    g[i][j] = p_l
13    return ans
14
15 def main():
16     n, m = list(map(int, input().split()))
17     g = []
18     for i in range(n):
19         row = [*input().strip()]
20         g.append(row)
21     dirs = [0, 1, 0, -1, 0]
22     for i in range(n):
23         for j in range(m):
24             if dfs(i, j, n, m, g, i, j, g[i][j], 0, dirs):
25                 print("Yes")
26                 sys.exit(0)
27     print("No")
28
29 if __name__ == "__main__":
30     main()

```

The Good

```

1 # Check if i, j coordinate is in boundaries of the matrix
2 def isInside(i, j, n, m):
3     if i < 0 or i >= n or j < 0 or j >= m:
4         return False
5     return True
6
7 # Runs dfs and returns true if there is a rectangle
8 def dfs(i, j, n, m, grid, start_i, start_j, start_letter, nodes_count, directions):
9
10    # Mark current cell as visited
11    prev_letter = grid[i][j]
12    grid[i][j] = '*'
13
14    answer = False
15
16    for direction in directions:
17        ni = i + direction[0]
18        nj = j + direction[1]
19
20        # If we hit to the start point, return true
21        if ni == start_i and nj == start_j and nodes_count + 1 >= 4:
22            return True
23
24        # If new explored cell is inside and satisfies condition, go to that cell
25        if isInside(ni, nj, n, m) and start_letter == grid[ni][nj]:
26            answer = dfs(ni, nj, n, m, grid, start_i, start_j, start_letter, nodes_count + 1, directions)
27            if answer:
28                break
29
30    # Revert back the letter, backtracking
31    grid[i][j] = prev_letter
32
33    return answer
34
35 def main():
36     n, m = list(map(int, input().split()))
37
38     grid = []
39     for _ in range(n):
40         row = [ *input().strip() ]
41         grid.append( row )
42
43     directions = [[0, 1], [1, 0], [-1, 0], [0, -1]]
44
45     for i in range(n):
46         for j in range(m):
47             is_possible = dfs(i, j, n, m, grid, i, j, grid[i][j], 0, directions)
48             if is_possible:
49                 print("Yes")
50                 sys.exit(0)
51
52     print("No")
53
54 if __name__ == "__main__":
55     main()

```

The Bad

```

1 def dfs(i, j, n, m, g, s_i, s_j, s_l, cnt, dirs):
2     p_l, g[i][j] = g[i][j], '*'
3     ans = False
4     for d in range(1, 5):
5         ni, nj = i + dirs[d], j + dirs[d - 1]
6         if ni == s_i and nj == s_j and cnt + 1 >= 4:
7             return True
8         if (ni >= 0 and ni < n and nj >= 0 and nj < m) and s_l == g[ni][nj]:
9             ans = dfs(ni, nj, n, m, g, s_i, s_j, s_l, cnt + 1, dirs)
10            if ans:
11                break
12    g[i][j] = p_l
13    return ans
14
15 def main():
16     n, m = list(map(int, input().split()))
17     g = []
18     for i in range(n):
19         row = [*input().strip()]
20         g.append(row)
21     dirs = [0, 1, 0, -1, 0]
22     for i in range(n):
23         for j in range(m):
24             if dfs(i, j, n, m, g, i, j, g[i][j], 0, dirs):
25                 print("Yes")
26                 sys.exit(0)
27     print("No")
28
29 if __name__ == "__main__":
30     main()

```

ThE Ugly.

```
#changes case, I like this function name better
def change_casing(str)str.swapcase; end
```

```
//quick bf interpreter
```

```
bF=(A,B,C,D,E,F,G,H)=>{for(E=[C=D=F=0],G='',H={'>':_=>++F<E.length||E.push(0),'<':_=>--F,'+':_=>++E[F]<256||E[F]=
String.fromCharCode(E[F]),',':_=>E[F]=B.charCodeAt(D++),'[':T=>{if(!E[F])for(T=1;T;) '['==A[++C]?++T:' '==A[C]&&--
for(T=1;T;)'] '==A[--C]?++T:'['==A[C]&&--T});C<A.length;++C)H[A[C]]());
return G}
```

```
var func = (function func(x) {
  collection = []
  for (let thing = 0; thing < 122; ++thing) {
    if (x[thing])
      collection.push('this is' + x[thing] + " ")
    else
      break;
  }
  collection
})
//don't forget 2 use recursion
#replace if/else w/nested ternaries!
```

```
#get divs
```

```
divs=lambda num: [x for x in range(2,int(num/2)+1) if num%x < 1] or
```

```
function memAlloc(banks) {
  var rec={},max=Math.max(...banks),maxi=banks.indexOf(max)
  rec[banks]=1
  while(1){
    var m = -1,mi = -1,il=maxi+banks.length,add=max/len|0
    banks[maxi]=0
    for(var i=maxi+1;i<=maxi+len;i++){
```

A2SV Example - 1

50. Pow(x, n)

Medium

👍 5925

💬 6459

❤️ Add to List

📄 Share

Implement `pow(x, n)`, which calculates `x` raised to the power `n` (i.e., x^n).

Example 1:

Input: `x = 2.00000`, `n = 10`

Output: `1024.00000`

Example 2:

Input: `x = 2.10000`, `n = 3`

Output: `9.26100`

Example 3:

Input: `x = 2.00000`, `n = -2`

Output: `0.25000`

Explanation: $2^{-2} = 1/2^2 = 1/4 = 0.25$

A2SV Example - 1

Code example taken from G31 - Submission

Bad Practice

```
1 ▾ class Solution:
2 ▾     def myPow(self, x: float, n: int) -> float:
3 ▾         if n == 0:
4 ▾             return 1
5 ▾         elif n % 2 == 0:
6 ▾             result = self.myPow(x,n//2)
7 ▾             return result ** 2
8 ▾         elif n == 1:
9 ▾             return x
10 ▾         elif n == -1:
11 ▾             return 1/x
12 ▾         else:
13 ▾             return self.myPow(x,n//2) * self.myPow(x,n-n//2)
14 ▾
```

Good Practice

```
1 ▾ class Solution:
2 ▾     def myPow(self, x: float, n: int) -> float:
3 ▾         ## getting power of x to absolute value of n
4 ▾         result = self.myPositivePow(x, abs(n))
5 ▾
6 ▾         ## if n is negative reverse the result
7 ▾         if(n < 0):
8 ▾             result = 1 / result
9 ▾
10 ▾         return result
11
12 ▾     def myPositivePow(self, x: float, n: int) -> float:
13 ▾         # any number rasied to 0 is 1.0
14 ▾         if(n == 0):
15 ▾             return 1.0
16
17 ▾         # do the half computation
18 ▾         halfPower = self.myPositivePow(x, n // 2)
19 ▾         fullPower = halfPower * halfPower
20
21 ▾         # if the power is odd multiply fullPower by x
22 ▾         if(n % 2 != 0):
23 ▾             fullPower *= x;
24
25 ▾         return fullPower
```

A2SV Example - 2

20. Valid Parentheses

Easy  16147  814  Add to List  Share

Given a string `s` containing just the characters `'('`, `)'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

```
Input: s = "()"
Output: true
```

Example 2:

```
Input: s = "()[]{}"
Output: true
```

Example 3:

```
Input: s = "]"
Output: false
```

A2SV Example - 2

Code example taken from G33 - Submission

Bad Practice

```
1 class Solution:
2     def isValid(self, s: str) -> bool:
3         dc = { '(': ')',
4                 '{': '}',
5                 '[': ']'
6             }
7
8         stack = []
9         for i in s:
10            if i in dc.keys():
11                stack.append(i)
12            else:
13                if len(stack) == 0:
14                    return False
15                op = stack.pop()
16
17                if dc[op] != i:
18                    return False
19            if len(stack) != 0:
20                return False
21            return True
```

God Practice

```
1 class Solution:
2     def isValid(self, s: str) -> bool:
3         # making valid pairs to identify them
4         validPairs = { '(': ')', '{': '}', '[': ']' }
5         stack = []
6
7         for char in s:
8             if char in validPairs.keys():
9                 # if the character is opening, add to stack
10                stack.append(char)
11
12            elif len(stack) == 0 or validPairs[stack.pop()] != char:
13                # if the character is closing,
14                # we need to have opening pairs in the stack
15                return False
16
17            # check if we have opening parenthesis left in the stack
18            return len(stack) == 0
```

Function Parameters

Good Practice

```
class Solution:
    def isValidHelper(self, current, lower_bound, upper_bound):
        if current == None:
            return True

        if not (lower_bound < current.val < upper_bound):
            return False

        left_answer = self.isValidHelper(current.left, lower_bound, current.val)
        right_answer = self.isValidHelper(current.right, current.val, upper_bound)

        return left_answer and right_answer

    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        return self.isValidHelper(root, float("-inf"), float("inf"))
```

Bad Practice

```
class Solution:
    def isValidHelper(self, root, lower_bound, upper_bound):
        if root == None:
            return True

        if not (lower_bound < root.val < upper_bound):
            return False

        return self.isValidHelper(root.left, lower_bound, root.val) and
self.isValidHelper(root.right, root.val, upper_bound)

    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        return self.isValidHelper(root, float("-inf"), float("inf"))
```

Other Recommendations

| Bad Practice

```
i=i+1
submitted +=1
x = x * 2 - 1
hypot2 = x * x + y * y
c = (a + b) * (a - b)
```

| Good Practice

```
i = i + 1
submitted += 1
x = x*2 - 1
hypot2 = x*x + y*y
c = (a+b) * (a-b)
```

Quote of the day

“Any fool can write code that a computer can understand.
Good programmers write code that humans can
understand.”

– Martin Fowler